

# Báo cáo cá nhân

Nguyễn Hữu Cảnh

*Trường Đại học Công nghệ (UET - VNU)*  
*Môn học: Kỹ thuật và Công nghệ dữ liệu lớn*

Ngày 3 tháng 10 năm 2026

## Phần 1: Technical Contribution

**1. Thay đổi hệ thống và trách nhiệm:** Nhiệm vụ kỹ thuật chính của em là thực hiện kịch bản đánh sập và kiểm thử khả năng phục hồi của hệ thống (Task 5), đồng thời kiểm tra chéo tính toàn vẹn của Volume Mount do Role B cấu hình.

**2. Hành vi hệ thống trước và sau thay đổi:**

- **Trước khi can thiệp:** Ứng dụng S3 đang chạy ổn định với 1 Pod, ổ cứng đính kèm (PVC object-data) ở trạng thái Bound. Toàn bộ 32 tệp dữ liệu tổng hợp (bench/r1-c1/\*.bin) được lập chỉ mục và truy cập thành công với mã băm toàn vẹn (tệp minh chứng: before-recovery.jsonl, pod-before.yaml, pvc-before.yaml).
- **Thay đổi hệ thống:** Ra lệnh xóa chủ động Pod lưu trữ (không xóa namespace, không xóa Deployment hay PVC).
- **Sau khi can thiệp:** Hệ thống Kubernetes tự động phát hiện Pod bị mất và khởi tạo một Pod thay thế mới với UID mới. Pod mới tự động đính kèm lại vào ổ cứng PVC cũ. Quá trình kiểm tra chéo xác nhận toàn bộ 32/32 tệp dữ liệu không bị thay đổi mã băm (tệp minh chứng: after-recovery.jsonl, pod-after.yaml, pvc-after.yaml). Các lớp bảo mật cũng được xác nhận vẫn giữ nguyên tác dụng qua các bài kiểm tra S06, S08, S10.

**3. Artifact:** Toàn bộ các tệp minh chứng được lưu trong thư mục evidence/ cá nhân, bao gồm các tệp YAML cấu hình trước/sau và tệp canary.jsonl ghi lại quá trình gián đoạn.

## Phần 2: Independent Check

**1. Tính toán thời gian phục hồi và xác minh độc lập:** Thông qua việc giám sát quá trình gián đoạn bằng tiến trình canary ping mỗi giây, em đã thu thập được chuỗi bằng chứng thô phản ánh thời gian chết của hệ thống.

**2. Dữ liệu thô trích xuất từ canary.jsonl:**

- t\_failure (Lỗi mạng đầu tiên): Mốc start\_s = **19.218s** (Lỗi EndpointConnectionError).
- Mất kết nối trong khoảng 20 giây với 14 yêu cầu liên tiếp bị từ chối (ok: false).
- t\_stable (Thành công ổn định trở lại): Mốc end\_s = **39.489s** (Kết quả trả về HTTP 200, ok: true).

**3. Kết quả tính toán:** Thời gian gián đoạn ( $T_{\text{observed}}$ ) = 39.489s - 19.218s = **20.271 giây**. Hệ thống đạt mục tiêu với thời gian phục hồi nhỏ hơn 120 giây.

**4. Giới hạn của các bằng chứng phục hồi:** Việc PVC giữ nguyên trạng thái và dữ liệu không bị mất chỉ chứng minh được khả năng re-mount ổ đĩa của nền tảng. Nó **không chứng minh** được những tính năng sau:

1. **Tính sẵn sàng cao:** Hệ thống thực tế vẫn mất khoảng 20 giây gián đoạn.
2. **Khả năng chịu lỗi khi sập nguồn:** Pod được xóa một cách có chủ đích và êm ái qua API, không mô phỏng được tình trạng hỏng hóc đột ngột khi S3 đang ghi tệp dở dang.
3. **Sao lưu và Phục hồi:** Toàn bộ dữ liệu vẫn nằm trên đúng một phân vùng lưu trữ vật lý duy nhất, không hề được nhân bản ra bên ngoài.
4. **Rủi ro trói buộc Node:** Ổ cứng PVC có thể đang bị gắn cứng với một Node vật lý. Việc toàn bộ Node đó bị đánh sập vẫn chưa được kiểm thử và có thể ngăn cản việc phục hồi.

## Phần 3: Reasoning & Accountability

**1. Tại sao các quyền observer của Kubernetes lại được phân tách khỏi quyền đọc S3, và tại sao một namespace operator lại nằm ngoài ranh giới cách ly được tuyên bố?**

Quyền observer của Kubernetes sử dụng hệ thống RBAC để kiểm soát các tương tác với API Server của cụm (xem trạng thái Pod, lấy sự kiện, đọc log). S3 data path không đi qua Kubernetes API, mà dùng xác thực thông qua Access Key/Secret Key riêng lẻ. Sự phân tách này giúp kiểm toán viên hạ tầng giám sát được trạng thái kỹ thuật mà không có khả năng giải mã hay đọc trộm dữ liệu bên trong ứng dụng. Một namespace operator nằm ngoài ranh giới cách ly vì họ có quyền truy cập vào các Secret và tạo Pod mới. Bằng đặc quyền này, họ có thể dễ dàng gắn S3 credential của nhóm khác vào một Pod tự tạo và phá vỡ các quy tắc bảo mật dữ liệu.

**2. Tại sao mức độ đồng thời bằng 4 lại có thể chậm hơn hoặc có độ trễ cao hơn mức độ đồng thời bằng 1?**

Với giới hạn khắt khe từ ResourceQuota (giới hạn CPU và RAM rất nhỏ), Pod lưu trữ S3 chỉ có một lượng tài nguyên cố định. Khi mức độ concurrency là 4, Pod phải chia nhỏ tài nguyên để duy trì 4 phiên kết nối mạng song song và thực thi thuật toán mã băm SHA-256 cho 4 tệp cùng lúc. Sự phân mảnh tài nguyên này tạo ra nghẽn cổ chai ở bộ vi xử lý và I/O đĩa cứng, khiến độ trễ của từng yêu cầu riêng lẻ bị kéo dài hơn so với việc tập trung xử lý dứt điểm từng tệp khi concurrency = 1.

**3. Tại sao việc phục hồi Pod lại không thiết lập được tính năng sao lưu, tính sẵn sàng cao hoặc lưu trữ bất buộc ?**

- **Không phải Backup:** Dữ liệu nguyên bản vẫn nằm cố định trên cùng một ổ đĩa PVC. Pod mới chỉ đơn thuần đọc lại dữ liệu cũ chứ không khôi phục từ một bản sao an toàn ở vị trí khác.
- **Không phải High Availability:** Trong khoảng thời gian thay thế Pod, ứng dụng đã từ chối 14 yêu cầu và gây ra downtime hơn 20 giây. High availability yêu cầu duy trì kết nối liên tục bằng cơ chế dự phòng multi-replica.

- **Không phải Enforced Retention:** Quá trình này không ngăn chặn được việc dữ liệu bị xóa. Một người dùng có quyền **Write** vẫn có thể sử dụng S3 API để xóa tệp hợp lệ bất cứ lúc nào.